Universidad de Las Palmas de Gran Canaria

Degree in Data Science and Engineering

BIG DATA — Academic Year 2026–2027

**Individual Assignment 03**

# Parallel Matrix Multiplication: Thread Scaling and Performance on an Apple M4

Empirical Research Report

María Báez Marrero

November 2026

Shared-memory parallelism in C and OpenMP; experimental data collected on Apple M4.

# Abstract

This study investigates the effect of OpenMP thread count on shared-memory dense matrix multiplication on an Apple M4 processor. Sequential and parallel C implementations use the same reordered `i-k-j` loop structure, 64-bit integers, contiguous row-major matrices, and deterministic inputs. Five matrix dimensions ($n = 500, 750, 1000, 1500, 2000$), four thread counts ($p = 1, 2, 4, 8$), and five measured repetitions per configuration yield 125 kernel timings. At $n = 2000$, the sequential median is 2.200 s and the eight-thread median is 0.796 s, giving 2.76× implementation speedup and 34.5% efficiency. A notable finding is that OpenMP with one thread takes 4.292 s. Diagnostic runs report about 24.2 versus 48.2 billion retired instructions, respectively. Clang optimization remarks identify loop interleaving in the sequential kernel but not the OpenMP kernel; when vectorization-related optimizations are disabled, single-run times converge to 4.272 and 4.254 s. These observations support an optimization-related explanation without proving the exact machine-code mechanism. Runtime variability, peak resident memory, correctness tests and experimental limitations are reported.


**Keywords:** matrix multiplication; OpenMP; multithreading; strong scaling; speedup; efficiency; benchmarking.

# 1 Introduction

Dense matrix multiplication is a common computational building block in numerical computing, scientific simulation, and data-intensive applications. For square matrices, the classical multiplication algorithm requires a number of arithmetic operations proportional to $n^3$. As matrix dimensions increase, it therefore becomes increasingly important to understand how implementation choices and available processing resources affect observed performance.

This assignment focuses on *shared-memory parallel execution*, rather than comparing programming languages or alternative mathematical algorithms. The experimental factor is the number of OpenMP threads; the primary response is elapsed kernel time, supported by speedup, parallel efficiency, observed variability, and representative process memory use. The work builds on a validated sequential reordered-loop method from the preceding optimisation assignment, with the parallel outer loop introduced as the main source-level change.

## 1.1 Research question

*How does increasing the number of OpenMP threads affect runtime, speedup, and efficiency for a reordered dense matrix multiplication on the tested Apple M4 machine?*

## 1.2 Hypotheses

The first hypothesis is that distributing independent output rows across multiple threads reduces elapsed time when the workload is sufficiently large. The second is that scaling is not perfectly linear: with increasing thread counts, scheduling, memory-system constraints, processor heterogeneity, and implementation-level effects may prevent efficiency from remaining at 100%. A third, exploratory question concerns whether the OpenMP implementation with one worker has a different cost from the separate sequential baseline.

### 1.3 Scope and contributions

The study measures five sizes and four thread configurations with equivalent logical inputs, and compares a separate sequential baseline with the OpenMP implementation. A targeted compiler-diagnostic experiment investigates the unexpectedly slow OpenMP one-thread case. The diagnostic examines optimization remarks and a build with vectorization-related optimizations disabled; it is not a complete SIMD extension, because it neither establishes the exact generated instructions nor supplies a repeated controlled SIMD performance study. Distributed execution and GPU acceleration are outside the scope. Conclusions apply only to the tested system and binaries.

## 2 Background

### 2.1 Classical multiplication and loop ordering

For matrices $A, B \in \mathbb{Z}^{n\times n}$, their product $C = AB$ is defined as

$$C_{ij} = \sum_{k=0}^{n-1} A_{ik}B_{kj}, \qquad 0 \le i, j < n. \tag{1}$$

The classical algorithm performs $\Theta(n^3)$ arithmetic operations. In row-major storage, the reordered loop nest `i-k-j` advances sequentially along the row of $B$ and the corresponding row of $C$ during the innermost loop. This can improve locality relative to an access pattern that repeatedly traverses columns of $B$; however, actual performance still depends on compiler transformations, caches, and the machine architecture.

### 2.2 Shared-memory parallelism

OpenMP provides directives and runtime functions for parallel programming on shared-memory systems [2]. In the implementation studied here, the outer `i` loop is distributed with `#pragma omp parallel for schedule(static)`. An iteration computes one complete output row, so two distinct iterations do not update the same elements of $C$. Arrays $A$ and $B$ are only read during multiplication. With the output initialised before entering the parallel loop, this row-based division permits safe concurrent execution without locks on matrix elements.

Static scheduling assigns chunks of the iteration space to available threads. It is appropriate for this experiment because each output row performs approximately the same amount of arithmetic work. Nevertheless, an equal iteration count does not imply equal execution time if processor cores differ in performance or the operating system schedules threads unevenly.

### 2.3 Speedup and efficiency

Let $T_{\text{seq}}(n)$ denote the median sequential kernel time and $T_p(n)$ the median runtime of the OpenMP implementation using $p$ threads. The reported speedup is

$$S(n, p) = \frac{T_{\text{seq}}(n)}{T_p(n)}. \tag{2}$$

The corresponding efficiency is

$$E(n,p) = \frac{S(n,p)}{p}. \tag{3}$$

Ideal linear scaling would have $S(n,p) = p$ and $E(n,p) = 1$. Amdahl's argument provides a conceptual explanation for diminishing returns when work cannot be perfectly parallelised [3]. The present results, however, also contain differences between compiled implementations, so it would be inappropriate to fit a serial fraction and treat it as a direct measurement of the algorithm's inherently sequential work.

### 2.4 Interpretation of one-thread OpenMP measurements

The OpenMP configuration with $p = 1$ remains an important experimental condition: it reveals the cost of the generated OpenMP path independently of actual parallel execution. It must not silently replace the distinct sequential baseline. Comparisons relative to the sequential baseline quantify *overall implementation speedup*; comparisons between OpenMP thread counts reveal scaling *within that implementation*. Keeping these two interpretations separate is essential for the discussion below.

## 3 Implementations and Correctness

### 3.1 Common representation and inputs

Matrices are contiguous arrays in row-major order with signed 64-bit integer elements (C type `long long`). The input matrices are allocated once per program execution and populated outside the timed section. A deterministic linear congruential generator uses seeds 42 and 4242 for $A$ and $B$, respectively; generated values lie in $\{0, 1, \ldots, 9\}$. Each configuration uses the same logical matrix entries for a given $n$.

### 3.2 Sequential baseline

The sequential kernel uses the loop order `i-k-j`. Its central structure is:

**Listing 1:** Sequential reordered multiplication (essential loop structure).

```
for (int i = 0; i < n; ++i) {
    for (int k = 0; k < n; ++k) {
        long long aik = A[(size_t)i*n + k];
        for (int j = 0; j < n; ++j) {
            C[(size_t)i*n + j] +=
                aik * B[(size_t)k*n + j];
        }
    }
}
```

Prior to this loop, the output array is zero-initialised. This zero-initialisation is included in the timed kernel boundary.

### 3.3 OpenMP implementation

The parallel kernel retains the same array representation and arithmetic expression, adding a directive before the outer loop:

**Listing 2:** Parallel outer-loop distribution used in the OpenMP kernel.

```
omp_set_dynamic(0);
omp_set_num_threads(threads);
#pragma omp parallel for schedule(static)
for (int i = 0; i < n; ++i) {
    for (int k = 0; k < n; ++k) {
        long long aik = A[(size_t)i*n + k];
        for (int j = 0; j < n; ++j) {
            C[(size_t)i*n + j] +=
                aik * B[(size_t)k*n + j];
        }
    }
}
```

The dynamic-team setting disables dynamic adjustment requested through the OpenMP runtime. The parallel loop is the only source-level shared-memory parallel construct in the measured kernel. The program does not call a multithreaded numerical library, avoiding an uncontrolled second pool of worker threads.

### 3.4 Correctness testing

Correctness was checked in two complementary ways. The benchmark stored a checksum of every output and verified checksum agreement across methods and thread counts for each matrix size. Separately, the dedicated correctness test, compared full outputs against expected values for a known $2 \times 2$ product, multiplication by the identity matrix, and multiplication by the zero matrix. The test exercised both sequential execution and OpenMP with 1, 2, 4, and 8 threads. All three named cases reported `PASS`.

For the known-answer case,

$$\begin{pmatrix} 1 & 2 \\ 3 & 4 \end{pmatrix} \begin{pmatrix} 5 & 6 \\ 7 & 8 \end{pmatrix} = \begin{pmatrix} 19 & 22 \\ 43 & 50 \end{pmatrix}.$$

Full element-wise comparison in these small cases is stronger than checksum agreement alone. The tests do not constitute a formal proof for arbitrary sizes, but they provide direct evidence for the required implementations. Integer multiplication introduces no floating-point rounding tolerance issue. The tested small input values and matrix sizes make overflow implausible, although overflow remains a general limitation of fixed-width arithmetic.

## 4 Experimental Methodology

### 4.1 Hardware and software environment

Table 1 lists the environment recorded for the study. The Apple M4 includes heterogeneous CPU cores; specifying 10 physical and 10 logical processors therefore does not imply that all worker threads run on cores with equal performance.

### 4.2 Factorial comparison

Five matrix sizes were tested: $n \in \{500, 750, 1000, 1500, 2000\}$. For each size, one independent sequential baseline and OpenMP configurations with $p \in \{1, 2, 4, 8\}$ were measured. With

**Table 1:** Recorded experimental environment.

| Property | Recorded value |
|---|---|
| CPU | Apple M4 |
| Physical CPU cores | 10 |
| Logical processors | 10 |
| Installed memory | 17,179,869,184 bytes (16 GiB) |
| Operating system | macOS 26.3.1 (build 25D771280a) |
| Compiler | Apple clang 21.0.0, ARM64 target |
| OpenMP runtime | `libomp` 23.1.5 |
| Python runtime | Python 3.13.0 |
| Compiler optimisation | `-O2` |
| OpenMP compilation | `-Xpreprocessor -fopenmp`, `-lomp` |
| CPU affinity | Not explicitly pinned; OS-managed scheduling |

five repetitions per condition, this yields $5 \times 5 \times 5 = 125$ recorded timings (25 configurations, five measurements each). The largest requested OpenMP team contained fewer threads than the machine's 10 logical processors, so the recorded configurations do not oversubscribe the hardware by thread count.

### 4.3 Timing boundary and procedure

The benchmark uses `clock_gettime(CLOCK_MONOTONIC)`. The timer surrounds output zero-initialisation and matrix multiplication, but excludes allocation, input generation, checksum calculation, process startup and CSV writing. Each invocation is a fresh process. The experiment did not report explicit warm-up iterations; this matters particularly when interpreting cold-state or scheduling effects. Raw timings were retained, and no sample was removed from the provided dataset.

For each configuration, the reported central value is the median of the five measured durations. Variability is measured by the interquartile range (IQR), computed by the analysis script using the `inclusive` quantile convention. The IQR is descriptive rather than a confidence interval. The experiment is intentionally bounded at $n = 2000$; no claim is made that this is the largest size supported by the hardware.

### 4.4 Memory measurement boundary

Peak process resident memory was recorded using `/usr/bin/time -l` on representative $n = 2000$ executions for sequential, OpenMP 1, OpenMP 4 and OpenMP 8. These measurements include process and runtime allocations in addition to the three matrix buffers. Each is a single diagnostic run, not a five-repetition memory distribution. Consequently, small differences should be treated descriptively and not as statistically established trends.

## 5 Measured Runtime Results

### 5.1 Median kernel times

Table 2 presents the median times from the complete benchmark. The baseline grows rapidly with $n$, as expected from the classical $\Theta(n^3)$ computation. All OpenMP configurations with

eight threads were faster than their lower-thread OpenMP counterparts at the reported medians.

**Table 2:** Median execution time in seconds (five measured repetitions per cell).

| Method | $n = 500$ | 750 | 1000 | 1500 | 2000 |
|---|---|---|---|---|---|
| Sequential | 0.037300 | 0.117621 | 0.276703 | 0.959420 | 2.199547 |
| OpenMP 1 | 0.072326 | 0.227936 | 0.543164 | 1.804705 | 4.291936 |
| OpenMP 2 | 0.041017 | 0.120385 | 0.273102 | 0.893722 | 2.156655 |
| OpenMP 4 | 0.020017 | 0.059605 | 0.138323 | 0.470890 | 1.134501 |
| OpenMP 8 | 0.010789 | 0.032948 | 0.084390 | 0.299031 | 0.796371 |

### 5.2 Scaling with matrix dimension

For the sequential baseline, the $n = 2000$ median is approximately 59.0 times the $n = 500$ median, while the ratio of dimensions is four and an ideal cubic ratio would be $4^3 = 64$. The rough agreement is consistent with a cubic algorithmic workload, although five sizes and one machine are insufficient to isolate cache behaviour or determine a precise asymptotic exponent. By contrast, OpenMP 8 changes from 0.010789 s to 0.796371 s across the same sizes, about 73.8 times; this ratio should not be interpreted as a change in algorithmic complexity, because parallel scheduling and hardware utilisation are not held constant across input dimensions.

### 5.3 Variability for the largest size

Table 3 gives the median, IQR and derived performance metrics at $n = 2000$. The OpenMP speedup and efficiency values are computed from the corresponding median timings, not from an average of per-run ratios.

**Table 3:** Complete headline comparison for $n = 2000$.

| Method | Median (s) | IQR (s) | Speedup | Efficiency |
|---|---|---|---|---|
| Sequential | 2.199547 | 0.008338 | 1.000 | 1.000 |
| OpenMP 1 | 4.291936 | 0.024532 | 0.512 | 0.512 |
| OpenMP 2 | 2.156655 | 0.030353 | 1.020 | 0.510 |
| OpenMP 4 | 1.134501 | 0.004823 | 1.939 | 0.485 |
| OpenMP 8 | 0.796371 | 0.011706 | 2.762 | 0.345 |

The eight-thread kernel achieves a reduction in median elapsed time of

$$100\left(1 - \frac{0.796371}{2.199547}\right) \approx 63.8\%. \tag{4}$$

This result is meaningful for the *tested pair of implementations*: it is not a claim of 8× acceleration or a guarantee that eight threads will always be the best choice.

## 6 Runtime Trends Across Thread Counts

Figures 1–3 show the increasing effectiveness of additional OpenMP threads at the smaller and intermediate matrix sizes. The displayed values use medians, avoiding direct reliance on individual unusually slow runs.

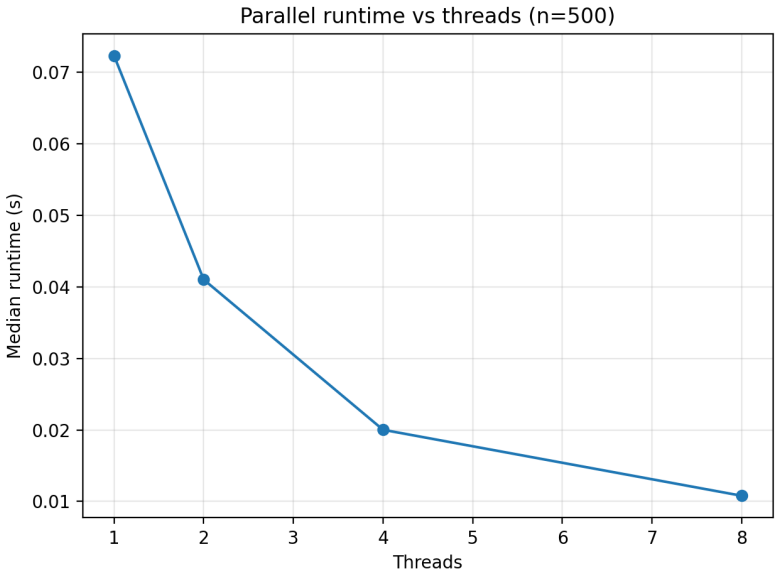



**Figure 1:** Median parallel runtime versus thread count for $n = 500$.

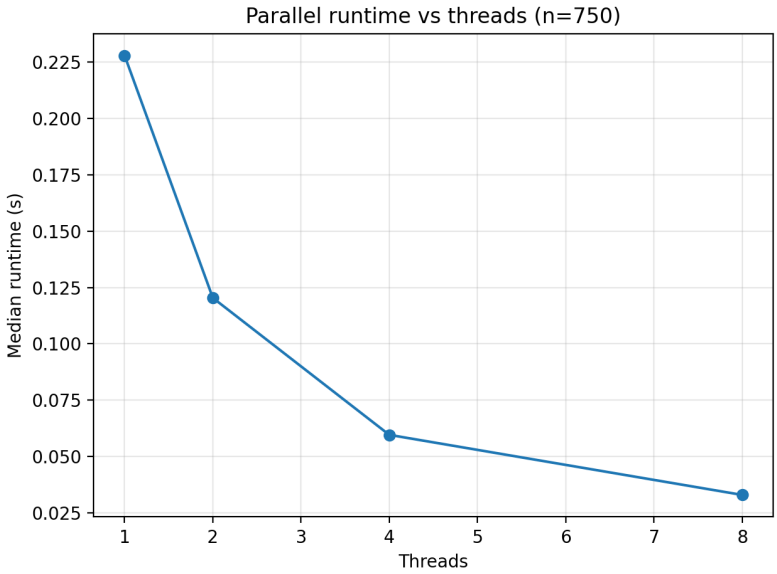



**Figure 2:** Median parallel runtime versus thread count for $n = 750$.

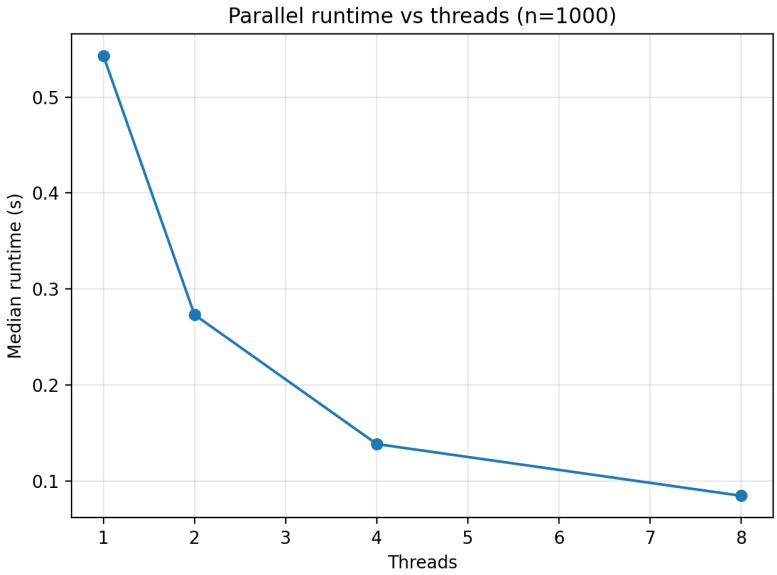



**Figure 3:** Median parallel runtime versus thread count for $n = 1000$.

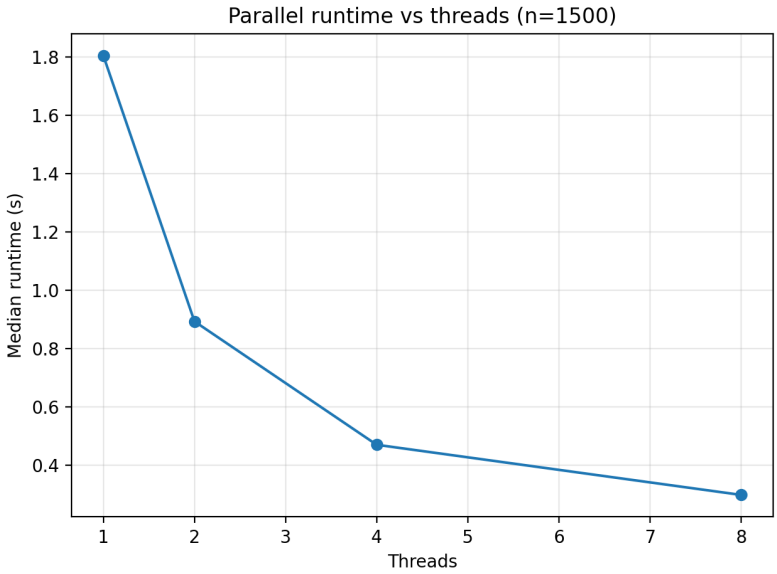



**Figure 4:** Median parallel runtime versus thread count for $n = 1500$.

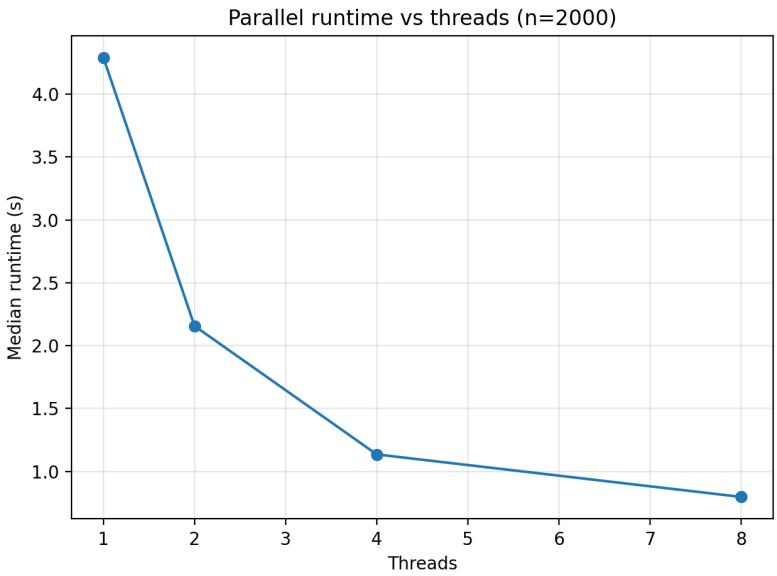



**Figure 5:** Median parallel runtime versus thread count for $n = 2000$.

At every tested dimension, using eight threads produces the shortest observed OpenMP median. At $n = 2000$, the OpenMP one-thread time is 4.292 s and the eight-thread time is 0.796 s: scaling *within the OpenMP family* is therefore about 5.39× from one to eight threads. The headline 2.76× speedup is lower because its denominator is the *faster separate sequential implementation*. These are different, valid ratios answering different questions; they must not be confused.

For $n = 500$, the sequential baseline takes 0.0373 s, compared with 0.0410 s for OpenMP with two threads, so two OpenMP threads do not yet improve on the separate sequential implementation. At $n = 2000$, the corresponding two-thread configuration is only marginally faster than the sequential baseline (2.157 s versus 2.200 s). The more convincing reductions relative to the sequential implementation occur with four and eight threads.

# 7 Speedup and Parallel Efficiency

## 7.1 Quantitative interpretation

For $n = 2000$, speedup is approximately 0.512, 1.020, 1.939 and 2.762 for one, two, four and eight OpenMP threads. The corresponding efficiencies are about 51.2%, 51.0%, 48.5% and 34.5%. Thus, while execution time improves as additional workers are used, observed scaling is sublinear relative to the independent sequential baseline. The increase from four to eight workers reduces runtime from 1.135 to 0.796 s, only about 1.42× rather than the ideal 2× reduction.

It is important not to interpret these numbers as an isolated measurement of the overhead of creating threads. Source-level algorithm equivalence does not guarantee equivalent machine instructions, vectorisation decisions or loop transformations. The substantially lower efficiency at one thread, combined with independent retired-instruction observations, suggests that the tested sequential and OpenMP binaries differ in more than runtime management alone.

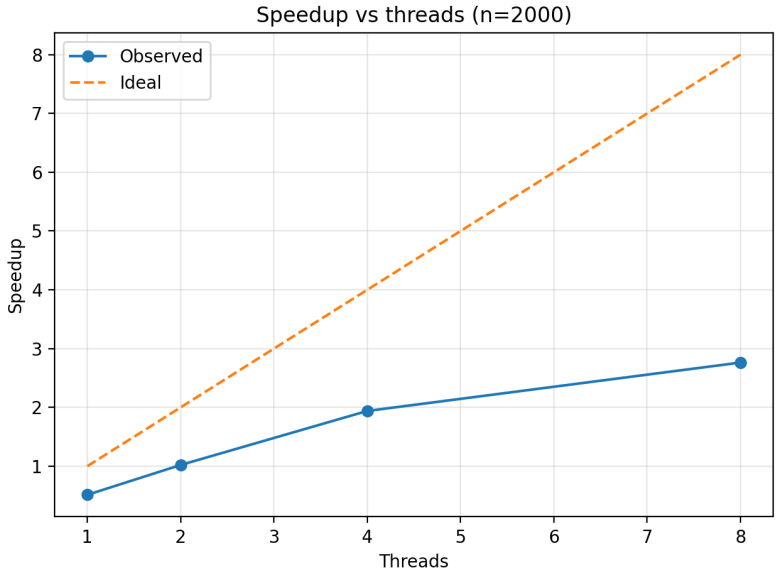



**Figure 6:** Observed speedup against the separate sequential baseline versus ideal linear speedup at $n = 2000$.

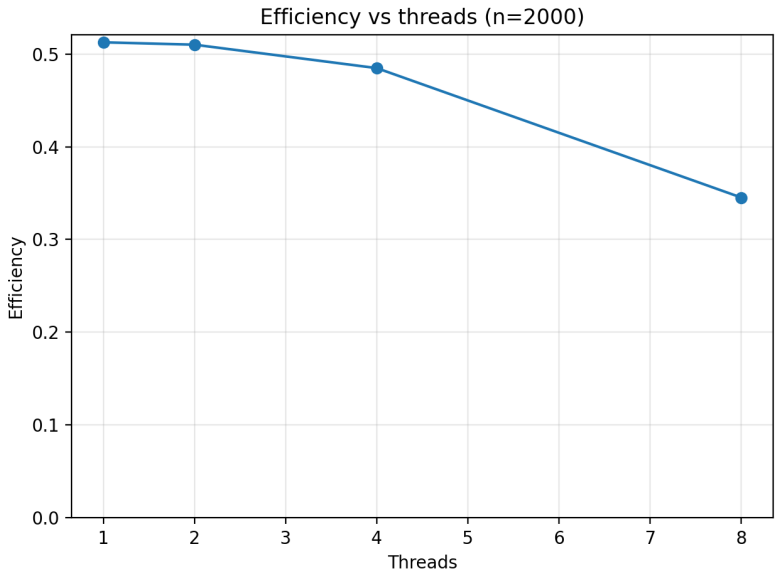



**Figure 7:** Efficiency $E = S/p$ for the $n = 2000$ experiment.

### 7.2 Compiler diagnostics versus a full SIMD extension

Although explicit SIMD/vectorisation is optional, compiler optimization diagnostics were used to investigate the one-thread anomaly. Clang's `-Rpass` remarks and a diagnostic build with `-fno-vectorize -fno-slp-vectorize` provide a partial controlled comparison of optimization settings. This is *not* a full SIMD study: no explicit SIMD intrinsics, assembly verification, or five-repetition comparison of vectorized and non-vectorized builds was performed. The original thread-scaling experiment remains the primary evidence.

## 8 Memory and Resource Use

**Table 4:** Single-run peak resident memory for $n = 2000$, measured with macOS `/usr/bin/time -l`.

| Configuration | Maximum resident set size (bytes) | MiB |
|---|---|---|
| Sequential | 97,566,720 | 93.05 |
| OpenMP 1 | 97,894,400 | 93.36 |
| OpenMP 4 | 98,025,472 | 93.48 |
| OpenMP 8 | 98,222,080 | 93.67 |

Three dense $2000 \times 2000$ matrices of eight-byte entries require $3 \times 2000^2 \times 8 = 96,000,000$ bytes (approximately 91.55 MiB) just for the stored elements. The process resident-memory measurements are higher because they include other process/runtime allocations. The difference between the sequential and eight-thread observations is $655,360$ bytes, or 0.625 MiB. That is a small descriptive increase and, because there is only one memory observation per method, it must not be presented as a statistically reliable trend. These RSS values are different from `peak memory footprint` also emitted by macOS `time`; this report consistently uses `maximum resident set size` for its memory table.

## 9 Discussion and Limitations

### 9.1 Why is OpenMP with one thread so slow?

Across the five tested sizes, OpenMP with one thread is slower than the sequential baseline. At $n = 2000$, the original medians are 4.292 and 2.200 s, respectively. A separate diagnostic run also reports about 48.203 billion retired instructions for OpenMP with one thread versus 24.219 billion for sequential execution. The four- and eight-thread runs report approximately 48.221 and 48.208 billion retired instructions, respectively. These counters suggest that the discrepancy involves more than a short OpenMP startup penalty.

Clang optimization diagnostics were collected using these flags:

```
-Rpass=loop-vectorize
-Rpass-missed=loop-vectorize
-Rpass-analysis=loop-vectorize
```

The sequential multiplication's inner loop received the remark *interleaved loop (interleaved count: 4)*, while the corresponding OpenMP loop received cost-model remarks that vectorization and interleaving were not beneficial. A second build disabled vectorization-related optimizations with `-fno-vectorize -fno-slp-vectorize`. Table 5 records the resulting times.

**Table 5:** Compiler diagnostic for $n = 2000$: original medians (five repetitions) versus single diagnostic runs with vectorization-related optimizations disabled.

| Configuration | Original median (s) | Diagnostic time (s) |
|---|---|---|
| Sequential | 2.200 | 4.272 |
| OpenMP, 1 thread | 4.292 | 4.254 |
| OpenMP, 4 threads | 1.135 | 1.176 |

Disabling these optimizations nearly doubles the sequential time but barely changes the OpenMP one-thread time. This is strong evidence that compiler optimization differences account for much of the observed gap. Importantly, the reported interleaving alone does not demonstrate the presence or absence of SIMD instructions, and the diagnostic runs have only one measurement each. For illustration, $4.272/1.176 \approx 3.63$ is a *single-run diagnostic ratio*, not a statistically validated replacement speedup or proof of identical generated machine code. The original 2.76× speedup remains a valid comparison between the actual tested binaries, but not an isolated estimate of threading benefit under otherwise identical compiled instructions.

### 9.2 Hardware heterogeneity and scheduling

An Apple M4 has different classes of CPU cores, so the 10 logical processors should not be modelled as 10 identical workers. The experiment did not record thread affinity or per-thread processor placement. Eight threads may use a mixture of core types, and the scheduling policy may change between independent runs. Memory bandwidth, cache sharing, and thermal or power states can also constrain improvement. While these are plausible mechanisms, none was directly measured here, so they are limitations and candidate explanations rather than established causes.

### 9.3 Benchmark design limitations

Each configuration was measured five times, enough for a descriptive comparison but not for narrow inferential confidence intervals. Matrix inputs were reproducible, but no explicit warm-up protocol was documented. The sequential and OpenMP results were collected in ordered groups, so time-varying machine conditions cannot be completely excluded. The timer includes output zero-initialisation but not allocation or checksum computation. Because peak RSS was measured only once per selected configuration, no memory dispersion statistic is available. Finally, the correctness tests cover known small inputs plus agreement of checksums for larger runs; they do not exhaustively verify all possible integer matrices.

### 9.4 What the evidence supports

The eight-thread configuration reduced median kernel time against the independent sequential baseline in all five tested sizes, although scaling relative to this baseline was sublinear. The large one-thread OpenMP deficit was reproducible; the compiler remarks, instruction counters and no-vectorization diagnostic jointly identify differing compiler optimizations as a plausible major contributor, without proving a specific SIMD instruction sequence. Peak resident memory was similar in the representative measurements. These conclusions are restricted to the measured binaries, inputs and Apple M4 hardware.

## 10 Conclusions

This study addressed how OpenMP thread count influences the performance of reordered dense matrix multiplication under controlled inputs on an Apple M4. The collected results show that increasing the number of OpenMP threads from one to eight consistently lowers the parallel implementation's median runtime for each tested matrix size. At $n = 2000$, the fastest tested configuration used eight threads and completed the kernel in a median of 0.796 s, compared with 2.200 s for the sequential baseline. The resulting 2.76× speedup and 34.5% efficiency demonstrate useful acceleration but not linear scaling.

The one-thread OpenMP version was nearly twice as slow as the separate sequential baseline. Compiler remarks showed an interleaved sequential loop, and a diagnostic build without vectorization-related optimizations produced similar single-run times for sequential and OpenMP one-thread execution (4.272 and 4.254 s). This supports an optimization-related explanation, while the exact machine-code mechanism remains unverified. Representative resident memory remained around 93–94 MiB for $n = 2000$. Known-answer, identity and zero-matrix tests passed, and large-run checksums agreed.

Further work could test all 10 available logical processors, alternate scheduling and thread-affinity policies, more repetitions in randomised configuration order, and assembly-level comparisons between the sequential and OpenMP builds. These extensions would refine causal interpretation, but they are not prerequisites for the primary experiment presented here.

## Reproducibility and Deliverables

The submission package should contain the C source file, correctness test, benchmark driver, analysis and plotting scripts, raw CSV, summary CSV, plots, this LaTeX source/PDF, and a README with commands. The project repository is available at https://github.com/mariabaezz/Assignment3.

**Listing 3:** Representative commands (run from the project root).

```
clang -O2 -Xpreprocessor -fopenmp src/matrix_parallel.c \
  -I"$(brew --prefix libomp)/include" \
  -L"$(brew --prefix libomp)/lib" \
  -lomp -o run_parallel
python3 benchmark/run_benchmark.py
python3 analysis/analyze.py
python3 analysis/plot_results.py
/usr/bin/time -l ./run_parallel omp 2000 8
```

## Declaration of AI Assistance

Generative AI assistance was used for interpretation support, and language/layout drafting of this report. The numerical tables and experimental claims are based on those observations, not on generated or simulated benchmark data.

## References

[1] Universidad de Las Palmas de Gran Canaria, *Individual Assignments: Matrix Multiplication*, Big Data, academic year 2026–2027, pp. 2–3, 6, 8–9.

[2] OpenMP Architecture Review Board, *OpenMP Application Programming Interface*, specifications and documentation. Available: https://www.openmp.org/specifications/.

[3] G. M. Amdahl, "Validity of the single processor approach to achieving large scale computing capabilities," in *AFIPS Conference Proceedings*, vol. 30, 1967, pp. 483–485. doi: 10.1145/1465482.1465560.

[4] Apple, *time(1)* command-line manual, macOS; command usage examined on the tested system.